Seminar: Image Segmentation Based on Visual Prompt

Group MNIST

VNUHCM - University of Science

December 16, 2025

Members

23122008 - Mai Duc Minh Huy
23122024 - Chung Tin Dat

Table of contents

1. Introduction
   - General Background
   - Motivation
   - Problem Statement
   - Contribution
2. Related Works
   - Traditional Methods
   - Deep Learning Methods
3. Methodology
   - Data Collection
   - Model Architecture
4. Implementation & Experiment
5. Conclusion
6. Discussion
7. References

General Background

Image segmentation is a fundamental task in computer vision that involves partitioning a digital image into multiple segments or sets of pixels, often corresponding to different objects or parts of objects. The goal is to simplify or change the representation of an image into something that is more meaningful and easier to analyze.

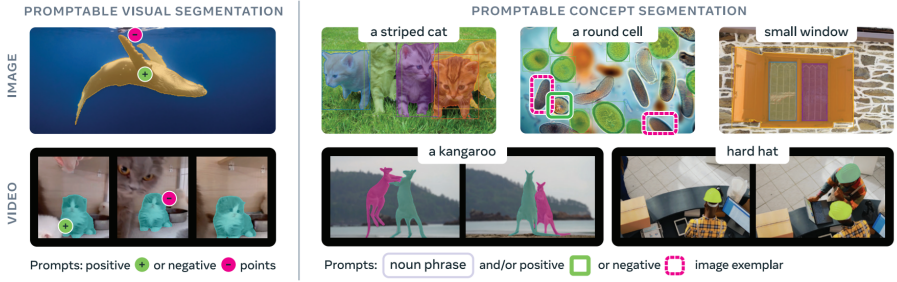
(a) Image (b) Semantic segmentation

(c) Instance segmentation (d) Panoptic segmentation

General Background

Visual prompts are user-provided graphical inputs—such as clicks, boxes, or lines that "tell" a computer vision algorithm where to focus or what specific object to process.

- **Points (Clicks):** The user clicks on a specific object (positive click) or background (negative click).
- **Bounding Boxes:** The user drags a rectangle around the object. This gives the algorithm a clear size and location limit.
- **Scribbles/Strokes:** The user draws a rough line over the object.



General Background

The history of this field can be broadly divided into two distinct eras.

- **Pre-Deep Learning Era (c. 1970s–2010s)**
- **Deep Learning Era (c. 2010s - Now)**

Pre-Deep Learning Era (c. 1970s-2010s)

Visual prompts function as constraints or "seeds." When a user marks a specific area, the algorithm treats those pixels as absolute truth—anchors that define the statistical profile (color intensity or texture) of the target object. The algorithm then propagates this information outwards to neighboring pixels, grouping them together based on how similar they are to the anchor points, stops only when it encounters a sharp change in contrast (an edge).

Pre-Deep Learning Era (c. 1970s-2010s)

Visual prompts function as constraints or "seeds." When a user marks a specific area, the algorithm treats those pixels as absolute truth—anchors that define the statistical profile (color intensity or texture) of the target object. The algorithm then propagates this information outwards to neighboring pixels, grouping them together based on how similar they are to the anchor points, stops only when it encounters a sharp change in contrast (an edge).

Prominent methods from this period include:

- Watershed
- Active Contour
- Grab Cut
- Region Growing

Pre-Deep Learning Era (c. 1970s-2010s)

Visual prompts function as constraints or "seeds." When a user marks a specific area, the algorithm treats those pixels as absolute truth—anchors that define the statistical profile (color intensity or texture) of the target object. The algorithm then propagates this information outwards to neighboring pixels, grouping them together based on how similar they are to the anchor points, stops only when it encounters a sharp change in contrast (an edge).

Prominent methods from this period include:

- Watershed
- Active Contour
- Grab Cut
- Region Growing

The prompt does not provide "understanding" of the object; it simply tells the math where to start calculating and what color values to look for nearby.

Deep Learning Era (c. 2010s)

Visual prompt is encoded into a high-dimensional vector and fused with the image features to act as a spatial query. Instead of just looking at neighbor pixel colors, the neural network uses the prompt to focus its "attention" mechanism on a specific region, combining the user's input with its own pre-trained knowledge of object shapes. For example, if a user clicks on a cat's ear, the model doesn't just select the ear's color; it recognizes the semantic pattern of a "cat" and predicts the entire animal.

Deep Learning Era (c. 2010s)

Visual prompt is encoded into a high-dimensional vector and fused with the image features to act as a spatial query. Instead of just looking at neighbor pixel colors, the neural network uses the prompt to focus its "attention" mechanism on a specific region, combining the user's input with its own pre-trained knowledge of object shapes. For example, if a user clicks on a cat's ear, the model doesn't just select the ear's color; it recognizes the semantic pattern of a "cat" and predicts the entire animal.
Key milestones include:

- Box Sup
- Scribble Sup
- Deep Extreme Cut
- Segment Anything

Motivation

The drive to advance image segmentation stems from both scientific inquiry and practical necessity.
Scientifically,

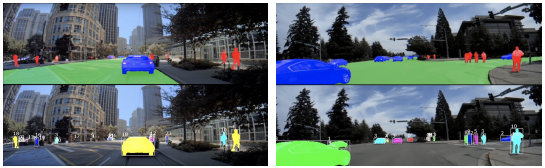
- Segmentation addresses a fundamental question in perception: while image classification tells us what is in an image, segmentation answers where it is, down to the pixel level; segmentation helps reveal how an image is organized by separating it into meaningful regions.
- Many methods in computer vision rely on segmented regions as inputs. Examples include object detection, recognition, tracking, 3D reconstruction, and medical image analysis. Better segmentation improves the performance of these tasks.

Motivation

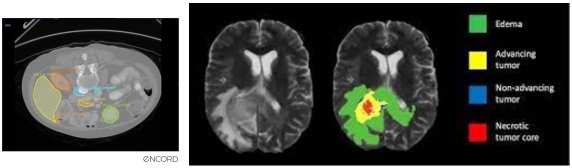
Practically, segmentation is an enabling technology for a multitude of real-world applications.

Motivation

Autonomous Driving

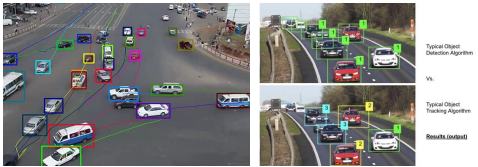


Medical Imaging

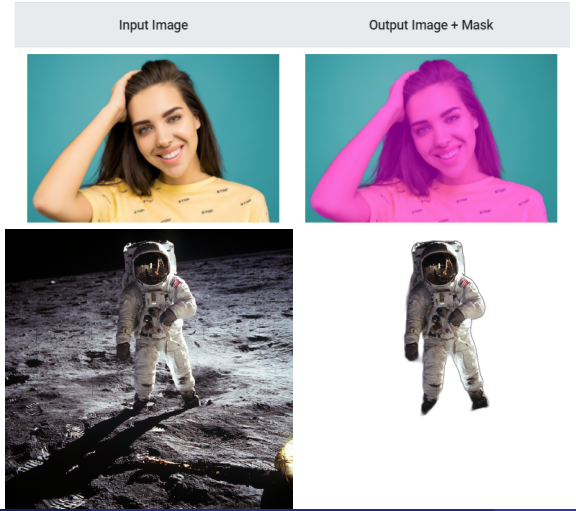


Surveillance

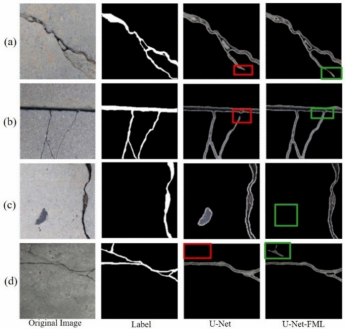
Object Tracking

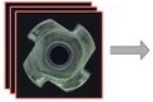


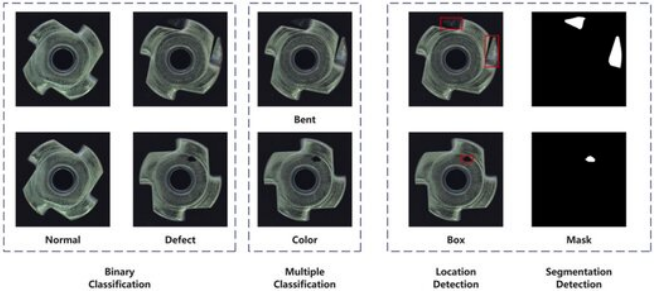
Digital Photograph



Industrial Inspection







Motivation

Segment Anything Model (SAM)

"In this work, our goal is to build a foundation model for image segmentation." [?]

With this model, we aim to solve a range of downstream segmentation problems on new data distributions using prompt engineering.

Problem Statement

Image segmentation is the task of grouping pixels that belong to a common semantic object and separating them from other objects and the background.
The core principle of segmentation is guided by two criteria: pixels within the same group should be as similar as possible, while pixels belonging to different groups should be as distinct as possible.
In essence, the task simulates the human cognitive ability to transform raw, unstructured visual data into a structured representation of distinct entities.

Problem Statement

Let $\Omega \subset R^2$ be the image domain and $I : \Omega \to R^k$ the image.
A segmentation is a partition

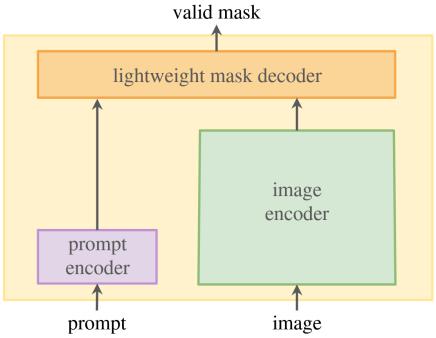
$$\Omega = \cup_{i=1}^{K} \Omega_i \qquad \Omega_i \cap \Omega_j = \emptyset \text{ for } i \neq j$$

such that pixels within each region share similar properties according to some criterion.

Problem Statement

For a modern promptable segmentation system, the problem can be formally defined by its inputs and outputs.

- **Input:** An image and a "visual prompt" that specifies the object of interest. This prompt can take various forms, such as a point (or multiple points) on the object, a bounding box enclosing it, or even a rough mask.
- **Output:** One or more segmentation masks for the object(s) indicated by the prompt, along with a confidence score for each mask.



Traditional Methods

Methods	**Mechanics**
Watershed	Flooding process on gradient topography with controlled basins.
Active Contour	Energy minimizing curve driven by internal smoothness and image forces.
Grab Cut	Local similarity expansion based on pixel intensity or color distance.
Region Growing	Probabilistic graph cut with iterative foreground background modeling.

Traditional Methods

Methods	**Methodology**
	Stage 1: Input preparation
Watershed	Image gradient, foreground and background markers.
Active Contour	Image as continuous energy field, initial contour.
Grab Cut	Image as graph with color distributions (GMMs), bounding box.
Region Growing	Image as pixel grid with feature vectors, seed pixels.

Traditional Methods

Methods	**Methodology**	
	Stage 2: Propagation/Optimization	**Stage 3: Final labeling**
Watershed	Flooding until basins meet.	Watershed lines form region boundaries.
Active Contour	Iterative energy minimization via curve evolution.	Final curve represents object boundary.
Grab Cut	Repeated min cut optimization with model updates.	Cut defines segmentation boundary.
Region Growing	Iterative neighbor expansion.	Region boundary from grown region.

Traditional Methods

Methods	**Contribution**
Watershed	Introduced topographic interpretation of images and marker controlled segmentation to reduce over segmentation.
Active Contour	Formalized segmentation as energy minimization over curves and influenced decades of variational methods.
Grab Cut	Demonstrated practical interactive segmentation with probabilistic modeling.
Region Growing	Established seed based interactive segmentation.

Traditional Methods

Methods	**Limitations**
Watershed	Sensitive to noise and gradient quality.
Active Contour	Struggles with concave boundaries and weak edges.
Grab Cut	Assumes color separability between foreground and background; fails when distributions overlap.
Region Growing	Highly sensitive to noise and intensity variation, leakage occurs at weak boundaries.

Deep Learning Methods

Technique	**Mechanics**
Box Sup	Unsupervised methods to generates hundreds of potential mask shapes (candidate segments), bounding box is used to filter these candidates.
Scribble Sup	Image is divided into super pixels, use Graph Cuts optimization to label the unmarked pixels.
Deep Extreme Cut	Heatmap from extreme points, CNN extracts feature specifically on the areas highlighted by the heatmap.
Segment Anything	Cross attention between image embedding and prompt embedding.

Deep Learning Methods

Technique	Methodology	
	Image representation	**Prompt encoding**
Box Sup	CNN feature maps extracted from region proposals and full images.	Bounding boxes converted into soft foreground background supervision using objectness and low level segmentation cues.
Scribble Sup	CNN feature maps over full image.	Scribbles transformed into dense label maps via graph based or CRF propagation.
Deep Extreme Cut	CNN encodes image appearance along with spatial context.	Extreme points encoded as distance maps concatenated with image channels.
Segment Anything	Vision transformer generates dense image embeddings.	Prompts encoded into tokens representing points, boxes, or masks.

Deep Learning Methods

Methods	**Methodology**
	Mask decoding
Box Sup	CNN trained on pseudo masks predicts dense semantic segmentation masks.
Scribble Sup	CNN outputs semantic masks learned from propagated pseudo labels.
Deep Extreme Cut	Single pass CNN inference produces object mask.
Segment Anything	Transformer based mask decoder predicts multiple candidate masks with confidence scores.

Deep Learning Methods

Technique	Contribution
Box Sup	It effectively combined traditional "region proposal" methods (unsupervised) with deep learning (supervised), allowing the network to learn shapes without ever seeing a ground truth mask.
Scribble Sup	It successfully integrated a Graphical Model (Graph Cut) into the deep learning pipeline. The graph propagates the scribble information to nearby pixels based on color/texture, creating a dense mask to train the CNN.
Deep Extreme Cut	It showed that annotating extreme points is 5x faster than drawing polygons but yields results that are nearly as good as fully supervised learning.
Segment Anything	First truly general purpose segmentation foundation model. Enables interactive segmentation, automatic segmentation, and box guided segmentation in one model.

Deep Learning Methods

Technique	Limitations
Box Sup	The model is limited by the quality of the candidate generation (MCG/Selective Search). If the proposal algorithm fails to generate a mask that fits the object, the CNN cannot learn it.
Scribble Sup	The "propagation" step relies on traditional color and texture similarity. If the object has low contrast, the graph cut fails to spread the label correctly.
Deep Extreme Cut	The user must provide exactly 4 points, and they must be the extreme edges. It struggles with objects that have disjoint parts because the extreme points might not capture the empty space in between efficiently.
Segment Anything	High computational cost during preprocessing due to large encoder. Not ideal for pixel perfect tasks such as thin structures or high precision medical boundaries without fine tuning. Mask quality depends on prompt placement.

Dataset SA-1B

The fundamental challenge in building a foundation model for segmentation is the lack of web-scale segmentation data.
Segmentation masks are not naturally occurring and must be manually created.
Traditional approaches would require paying annotators to label billions of masks-economically infeasible and temporally prohibitive.

Dataset SA-1B

SAM Data Engine:

1. Assisted Manual Annotation (4.3M Masks from 120K Images)
2. Semi-Automatic Annotation (5.9M Additional Masks, 180K Images)
3. Fully Automatic Annotation (1.1B Masks, 11M Images)

Dataset SA-1B

1. Assisted Manual Annotation (4.3M Masks from 120K Images):
Workflow:

1. Professional annotators view an image in a web-based interface.
2. They click foreground/background points to indicate an object of interest.
3. SAM's pre-computed image embedding (from MAE-ViT-H) is queried with these prompts.
4. The lightweight Prompt Encoder and Mask Decoder run in real-time (< 50ms), generating an initial mask prediction.
5. Annotators refine the mask using pixel-precise brush and eraser tools.
6. The final mask is saved.

Dataset SA-1B

Quantitative Dynamics:

- **Initial annotation time:** 34 seconds per mask (comparable to COCO annotation standards).
- **Model capacity progression:** ViT-B $\rightarrow$ ViT-H during this stage (1 retraining cycle per major improvement).
- **Average masks per image:** 20 $\rightarrow$ 44 (increased diversity as model improved).
- **Total masks collected:** 4.3M from 120K images.
- **Retraining cycles:** 6 full model retraining iterations.

Efficiency Gain Analysis:
The 34 → 14 second progression represents a **6.5× speedup over COCO standards** (34 seconds here vs. 14 seconds for bounding boxes). This exponential improvement in annotation velocity is crucial for economic feasibility at billion-scale.

2. Semi-Automatic Annotation (5.9M Additional Masks, 180K Images):
Workflow:

1. Train a generic object detector on all masks from Stage 1. This detector outputs bounding boxes regardless of semantic class (object-agnostic proposal generation).
2. SAM automatically segments high-confidence regions identified by this detector, generating candidate masks.
3. Annotators are presented with images pre-filled with these automatic masks.
4. Annotators' task: Label only the *unannotated* objects (those not covered by the automatic masks).
5. This forces annotators to focus on smaller, more occluded, or semantically ambiguous objects.

Dataset SA-1B

Quantitative Dynamics:

- **Annotation time (excluding automatic masks):** $\sim$34 seconds per mask (unchanged; challenging objects require the same effort).
- **Average masks per image:** 44 $\rightarrow$ 72 (including automatic masks; diversity increased substantially).
- **New masks collected:** 5.9M, raising the cumulative total to 10.2M masks.
- **Retraining cycles:** 5 full model retraining iterations.

Efficiency Gain Analysis:
The $34 \rightarrow 14$ second progression represents a **6.5× speedup over COCO standards** (34 seconds here vs. 14 seconds for bounding boxes). This exponential improvement in annotation velocity is crucial for economic feasibility at billion-scale.

Why stage 2 is needed?
By automatically handling easy cases and forcing annotators to label hard cases, the model learns a more balanced representation. This is empirically validated: the model's ability to handle edge cases improved significantly from Stage 1 to Stage 2, as evidenced by the diversity metrics.

3. Fully Automatic Annotation (1.1B Masks, 11M Images):
Systematically prompt the model with a regular spatial grid of points (32×32), covering the entire image. Workflow:

1. Each point is passed as a prompt to the Prompt Encoder.
2. For each point, SAM predicts **three masks simultaneously**.
3. **Theoretical maximum:** 1024 points $\times$ 3 masks per point $=$ 3072 masks per image.

Dataset SA-1B

Filtering, Deduplication, Non-Maximum Suppression

1. IoU Scoring and Confidence Thresholding
2. Stability Filtering
3. Non-Maximum Suppression (NMS)

IoU Scoring and Confidence Thresholding

- For each of the three masks predicted from a grid point, SAM outputs a **predicted IoU score** (estimated mask quality).
- **Threshold:** Only masks with **predicted IoU > 0.88** are retained.
- **Rationale:** High IoU threshold ensures only high-quality masks enter the deduplication pipeline.
- **Expected reduction:** From 3072 theoretical outputs, this threshold alone eliminates $\sim$80% of predictions (keeping only predictions the model is highly confident about).

Stability Filtering

- A mask is **stable** if small perturbations in the model's confidence threshold (e.g., changing the foreground probability threshold from 0.5 to $0.5 \pm \epsilon$) produce similar masks.
- **Operationally:** Threshold the mask's probability map at both 0.5 and 0.5, and compute the IoU between the two resulting binary masks.
- If this "stability IoU" is high (> 0.95), the mask is kept; otherwise, it is discarded.
- **Why This Matters:** Masks that are brittle (sensitive to small threshold changes) are often segmentation errors or include spurious boundaries. Stability filtering removes low-confidence predictions and keeps robust ones.
- **Further reduction:** $\sim$50% of remaining masks fail stability checks.

Non-Maximum Suppression

1. Sort masks by predicted IoU score.
2. Select the highest-scoring mask m_1.
3. Foa all remaining masks m_i:
   - Compute $\text{IoU}(m_1, m_i) = \frac{|m_1 \cap m_i|}{|m_1 \cup m_i|}$ (Jaccard index).
   - If $\text{IoU}(m_1, m_i) > \text{thresh}_{\text{IoU}}$ (typically 0.7), discard m_i (it is a near-duplicate).
   - If $\text{IoU}(m_1, m_i) \leq \text{thresh}_{\text{IoU}}$, retain m_i (it is a distinct object).
4. Repeat on remaining unprocessed masks.

Dataset SA-1B

Scale Reduction Summary:

- **Start:** 3072 theoretical masks (32×32 grid $\times$ 3 outputs per grid cell).
- **After IoU filtering (> 0.88):** $\sim$600 masks (20% remain).
- **After stability filtering:** $\sim$300 masks (50% of IoU-filtered).
- **After NMS (thresh_IoU $= 0.7$):** $\sim$ **100** masks (final).

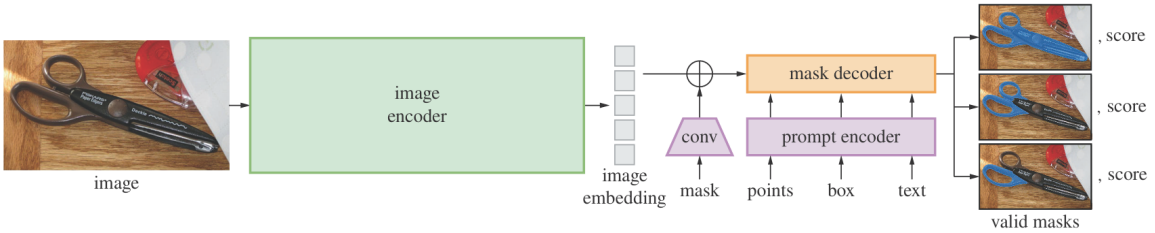
Final Calculation:

$$\text{Total SA-1B Masks} = 11\text{M images} \times 100 \text{ masks/image} = 1.1 \text{ B masks}$$

Why stage 3 is needed?
A regular grid ensures comprehensive spatial coverage. If an object's boundary passes through or near a grid point, the model will capture it from that point. Objects of varying sizes are covered because the grid is at a fixed, moderate resolution. This is fundamentally different from random point sampling, which can miss objects entirely.

Why 3 outputs per prompt?
Empirical analysis during SAM's development showed that image regions typically have at most three levels of semantic hierarchy (whole > part > sub-part). Predicting three masks covers all common ambiguity scenarios without excessive computation.

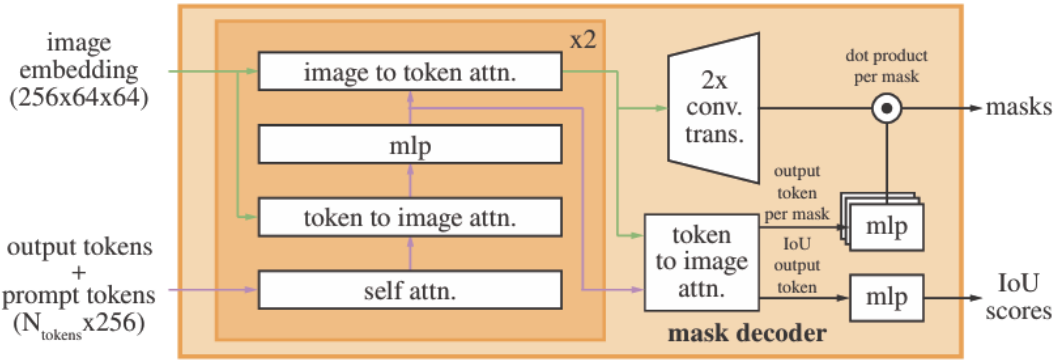
Segment Anything Model



- Image Encoder: MAE pre-trained Vision Transformer (ViT).
- Prompt Encoder:
  - Point, box: Positional encoding summed with learned embeddings for each prompt type.
  - Mask: Convolution and summed element-wise with the image embedding.

Segment Anything Model

- Mask Decoder:



Segment Anything Model

Output token is an embedding vector acts as a query for the final mask prediction (analogous to CLS token in BERT or ViT).
This token gether information about the prompt embeddings and image embeddings to form semantic information required to segment the target object.

Segment Anything Model

- Image Encoder: Fine-tune its weights to adapt its visual features to be queryable.
- Mask Decoder: Transformer layers, MLP, upscaling convolutions.
- Prompt Encoder: Convolutional layers for positional encodings, vectors for "Foreground", "Background", Top-left", "Bottom-right", Not-a-point".
- Output token: 4 vectors (3 for multi-mask outputs: whole, part, sub-part, 1 for IoU confidence prediction).

Loss Function

A linear combination of focal loss and dice loss in a 20:1 ratio of focal loss to dice loss.

- Local Loss: Focus to learning energy to hard pixels.

$$FocalLoss = -\Sigma_{i=1}^{n}(i - p_i)^{\gamma} log_b(p_i)$$

- Dice Loss: Differentiable approximation of IoU.

$$DiceLoss = \frac{2 * Intersection}{Prediction + Ground\_Truth}$$

References I

Chen, J., Lu, Y., Yu, Q., Luo, X., Adeli, E., Wang, Y., Lu, L., Yuille, A. L., and Zhou, Y. (2021).
Transunet: Transformers make strong encoders for medical image segmentation.
arXiv preprint arXiv:2102.04306.

Comaniciu, D. and Meer, P. (2002).
Mean shift: a robust approach toward feature space analysis.
IEEE Transactions on Pattern Analysis and Machine Intelligence, 24(5):603–619.

Hartigan, J. A. and Wong, M. A. (2018).
A k-means clustering algorithm.
Journal of the Royal Statistical Society Series C: Applied Statistics, 28(1):100–108.

References II

He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017).
Mask r-cnn.
In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969.

Nameirakpam, D., Singh, K., and Chanu, J. (2015).
Image segmentation using k -means clustering algorithm and subtractive clustering algorithm.
Procedia Computer Science, 54:764–771.